SCS2314 Lab 1

Multithreaded Chat Server with Java Sockets

Student index: _____ Date: 13 October 2026

Method and implementation

The supplied starter code was completed and tested in a Linux cloud environment using Temurin JDK 21.0.8. The experiments used localhost:5000. Compilation used javac --release 21; the final Maven build was also checked. Eclipse was not used. TCP peers were driven by an automated harness, alongside the supplied ChatClient, CounterDemo and LoadTester programs. All values below are measured results, not example values.

Separate unsafe, list-fixed and fully fixed source versions were compiled for Part 4. A fresh server was started for each load-test run to isolate failures. The submitted ChatServer uses a ten-worker pool by default; its optional --thread-per-client mode retains the Part 4 workflow.

Part 1: Single-threaded echo server

The first client received its welcome and an echo reply. A second TCP connection succeeded, but received neither a welcome nor an echo while the first client remained connected. After the first client sent bye, the second received its welcome and echo: second message. The main thread was blocked in handle(), reading the first client, and could not return to accept(). A successful TCP connection therefore did not mean the application was already serving that client.

Parts 2 and 3: Concurrent echo and chat

ThreadedEchoServer returned the expected echo replies to three simultaneously open clients. Server logs identified three different workers: Thread-0, Thread-2 and Thread-1. Each worker blocks waiting for its own client; the main thread continues accepting connections.

The five ClientHandler TODOs register clients, broadcast messages, implement /list, count chat messages and remove disconnected clients. Three chat peers verified joins, delivery to all recipients, online: Alice, Bob, Carol, stats: 1 messages and departure on /quit. The provided Java ChatClient also sent a message successfully. A generic TCP peer could participate because the protocol is UTF-8 text lines, independent of the client implementation language.

Part 4a: Counter experiment

Expected total in every run: 50 threads x 100,000 increments = 5,000,000.

Run	Unsafe actual	Lost updates	AtomicInteger actual
1	2,866,621	2,133,379	5,000,000
2	2,181,347	2,818,653	5,000,000
3	2,445,166	2,554,834	5,000,000

Think - lost update: count++ reads, adds and writes. If two threads both read 10, each computes 11 and writes 11; two increments produce only one increase. volatile provides visibility and ordering, but the compound increment is still not atomic. AtomicInteger.incrementAndGet() makes each increment atomic, and get() reads the result. All three corrected runs produced 5,000,000 with zero lost updates.

Part 4b: Chat server under load

Each test used the supplied default LoadTester: 50 clients x 1,000 messages, with five churn threads repeatedly joining and leaving. Expected count per run: 50,000. The server count is the difference between /stats readings before and after the run; commands and join/leave announcements are not counted.

Version	Run 1	Run 2	Run 3	Server exceptions
Unsafe list and counter	No reply	540	154	CME in all runs; NPE also in run 1
List fixed only	49,921	49,902	49,934	None
List and counter fixed	50,000	50,000	50,000	None

CME = ConcurrentModificationException; NPE = NullPointerException. "No reply" is an unavailable count, not zero. In unsafe run 1, /stats failed to reply and all 50 load clients reported write errors. Runs 2 and 3 reported 46 and 47 clients with write errors, respectively. These were failed experiments, despite LoadTester returning exit code 0.

Observed effect of each fix

The unsafe ArrayList was modified while broadcast() iterated over it. ConcurrentModificationException terminated client handlers; run 1 also logged a null list entry in broadcast(), causing NullPointerException. Affected handlers closed their sockets, so many messages were never processed and the statistics request could fail.

CopyOnWriteArrayList removed the iteration exceptions and client write errors, but the plain integer still lost 79, 98 and 66 increments. After replacing messageCount with AtomicInteger and updating both incrementAndGet() and get(), all three tests counted 50,000, with no exceptions or tester warnings. Their measured durations were 4,243 ms, 4,380 ms and 3,968 ms. A further test of the actual submitted source in --thread-per-client mode also counted 50,000 with zero lost messages.

Think: Why do exceptions reduce the count so much?

The read loop runs inside a client handler. An unchecked exception exits that loop and triggers finally, which disconnects the client. The worker stops reading the remaining queued messages, and later writes from that client can fail. This loses processing of whole message sequences, in addition to any increments lost through counter races.

Think: Why does a successful test not prove correctness?

A race depends on the timing and interleaving of threads. A particular run may avoid overlapping increments and appear correct, especially when other work delays the threads. Other CPU loads or client timing can expose it. Repeated testing provides evidence, but correctness also requires a concurrency argument: atomic increments and safe list iteration remove the specific unsafe operations.

Think: Why synchronize send(), and can message order differ?

PrintWriter.println() already locks internally, so a single-line send normally remains intact. Declaring send() synchronized makes the per-client serialization rule explicit and protects future changes that might perform several writer operations. It does not impose a total order across all clients. Two broadcasts can interleave their visits to different client handlers, allowing one recipient to receive A then B while another receives B then A. A single broadcast dispatcher or sequencing protocol would be needed for a consistent global order.

Parts 5 and 6: Capacity and failure

Part 5: Ten-worker thread pool

ChatServer creates Executors.newFixedThreadPool(10) before its accept loop and submits each ClientHandler to it. With LoadTester --clients 12 --hold, exactly ten hold handlers were logged as connected before release. A thirteenth TCP peer connected and sent /list, but received no response while all workers were occupied. After Enter released the hold clients, both queued hold handlers started and the waiting peer received online: hold10, hold11, WaitingClient. The hold program exited successfully.

Think: Thread-per-client versus a fixed pool

One thread per client is simple and lets every connected client have a handler immediately, but idle blocking clients still consume thread resources. Creating 100,000 platform threads can exhaust memory or operating-system limits and adds scheduling overhead. A fixed pool caps active workers, but long-lived idle clients occupy all ten slots and prevent queued clients from being served. The default fixed pool has an unbounded queue: pending sockets and tasks still consume memory. A bounded ThreadPoolExecutor queue, explicit admission limit and rejection handler that closes rejected sockets could refuse excess work. This lab demonstrates the pool limit; it does not implement that additional production policy.

Part 6: Abrupt client and server failure

Alice, Bob and the supplied Java ChatClient named CrashClient were connected. CrashClient was forcibly killed without /quit. Both survivors received * CrashClient left, /list returned online: Alice, Bob, and both still exchanged a subsequent message. This verifies cleanup on unexpected process termination rather than only the normal quit path. disconnect() runs from finally, removes the registered handler and closes the socket in its own finally block.

Stopping the server closed the remaining TCP connections. In a separate check, the supplied Java ChatClient printed -- disconnected from server -- and exited. These observations cover detectable local socket closure; they do not establish immediate detection of every remote network outage.

Think: Can the server distinguish a crashed client from a quiet one?

An orderly TCP close produces end-of-stream and some failures produce an I/O error. However, a silent network failure or unreachable machine can leave a connection apparently open, with readLine() waiting just as it would for a quiet client. Absence of messages alone cannot distinguish the cases reliably. Application heartbeats and read deadlines can flag a client as unresponsive, but a timeout is a suspicion of failure: delay or congestion can also trigger it. Cleanup must run whenever disconnection is detected, and other clients should remain usable.

Build and reproduce the submitted code

Use JDK 21. From the directory containing pom.xml, run mvn compile. For the pool demonstration, start the default server:

```
java -cp target/classes chat.ChatServer 5000
```

For Part 4 load testing, use a separate server run with:

```
java -cp target/classes chat.ChatServer 5000 --thread-per-client
```

```
java -cp target/classes chat.LoadTester
```

Stop the previous server before starting another on port 5000. Using the ten-worker pool for the 50-client stress experiment changes the experiment: clients queue behind occupied workers. CounterDemo in the submission is the corrected atomic version. Unsafe and intermediate measurements came from separate preserved experiment versions.

Submission contents and scope

The submission ZIP contains this three-page report, src/ (including the original echo server, ThreadedEchoServer and completed chat classes), and the supplied pom.xml. It excludes target/. The original Git repository was left unchanged. Measurements were generated by automated cloud execution; they were not collected through Eclipse or manually on the student's laptop. Add the student index and rename the ZIP before submission.